Data Science Playbook for Researchers

Bevin Manuelpillai

Contents

My Philosophy of Coding as a Researcher **4**

The Motivation to Learn to Code as a Researcher . . . 4

How Scientists Should Approach Coding . . . 4

The Doors Coding Opens for Researchers . . . 4

Why Coding is Fun . . . 4

Forest vs. Trees . . . 5

Programming and Data Science Learning Path **6**

Phase 1: Getting Started . . . 6

Phase 2: Writing Cleaner Code . . . 6

Phase 3: Apply Your Skills to Research . . . 7

Phase 4: Optimization and Customization . . . 7

Data Manipulation in R and Python **8**

Cleaning and Visualizing Data . . . 8

tidyverse (R) . . . 8

pandas (Python) . . . 8

Writing Functions in R **10**

Topics Covered . . . 10

Why Use Functions . . . 10

Control Flow in R . . . 11

Loops and Iteration . . . 11

Working with Packages . . . 12

Debugging in R **13**

Learn to Debug R Code . . . 13

Debugging Functions . . . 13

Version Control Your Code **14**

Why You Should Learn Git and GitHub . . . 14

General Git Workflow . . . 14

Four Levels of Version Control . . . 14

Level 1: Getting Started with GitHub . . . 15

Level 2: Working with Branches . . . 15

Level 3: Advanced Git Features . . . 17

Level 4: Continuous Integration . . . 17

Command Line Tools **19**

Why Learn Command-Line Tools . . . 19

Getting Good with Your Terminal . . . 19

File Management . . . 20

Package Management . . . 21

Tips for Effective Terminal Use . . . 21

Introduction to Shell Scripts . . . 21

Python Environments **22**

Going from R to Python . . . 22

Poetry Installation . . . 22

General Workflow Using Poetry . . . 22

Use Conda to Manage Dependencies Beyond Python . . . 23

Tools for Machine Learning **24**

scikit-learn . . . 24

tidymodels . . . 24

Types of Machine Learning Algorithms . . . 24

Data Splitting . . . 24

Choosing the Right ML Algorithm . . . 25

Neural Networks . . . 25

Basic Hyperparameters in Neural Networks . . . 25

Getting Started with Bayesian Inference **27**

Recommended Resources to Learn Bayesian Inference . . . 27

Programmer's Toolbox **28**

New Tools . . . 28

Why Learn VS Code . . . 28

What Are Vim Motions? . . . 28

Why Learn Vim Motions . . . 28

Autoformat Code in R . . . 29

Useful Tools for Developers . . . 29

Further Learning **30**

Useful Courses . . . 30

Fun Content . . . 30

My Philosophy of Coding as a Researcher

Coding as a Tool for Research

The Motivation to Learn to Code as a Researcher

The Start Is The Hardest Part

When you first start to program, it can feel really overwhelming. You aren't comfortable with the language, and you have to look up everything before you can do what you want. You'll see more experienced programmers doing things a lot faster and in ways you've never seen before. It can be discouraging, but rest assured this will be the hardest part of your programming journey. Once you gain experience and confidence as a programmer, you will feel like you can tackle any problem.

How Scientists Should Approach Coding

Video: How to learn programming - Advice for scientists | Clara Sousa-Silva and Lex Fridman

In this clip from the Lex Fridman Podcast, quantum astrochemist Clara Sousa-Silva shares her advice on how scientists should approach learning to program.

The Doors Coding Opens for Researchers

Developing your coding skills opens a lot of doors when you go to analyze your data. If you have lots of data, you probably will not be able to run your analysis on your local computer due to hardware constraints. Learning the basics of computing clusters or cloud computing will allow you to tackle this problem, which is especially important for computationally demanding methods such as Bayesian inference.

Why Coding is Fun

Coming from a STEM background, one of my favorite parts about programming is how you get instant feedback from running your code. Either your code runs or you get an error message. Every time your code runs, you get a feeling of accomplishment that drives you to overcome the next bug. Every time you squash a bug, you improve your ability to diagnose and fix the next one.

I also love the community of developers and researchers who support you through the material they share online. You can look at other people's projects or online tutorials for inspiration. I highly recommend

looking for course websites that professors post online, as they are tailored for researchers and often include homework assignments to reinforce your learning. Most importantly, you learn by doing, so pick projects you enjoy.

Forest vs. Trees

Don't miss the forest for the trees. I think coding is especially fun for scientists because it isn't our main job. We use code as a tool to answer the questions that interest us; our goal is not to write the most optimized or well-written. You may become really good at coding as a byproduct of tackling your research questions, but coding is just a means to an end. Write code first, then optimize it later if needed. That is not to say your code shouldn't be readable – it should still be well documented so that others can work on it too.

I include a lot of tools that make coding easier and more fun, but when you are first starting, focus on getting the basics down. That comes with time and practice. I recommend following one course and then spending most of your time on projects.

Programming and Data Science Learning Path

Four Phases Building Foundational Programming Skills

Phase 1: Getting Started

Get comfortable working with data. You should be able to manipulate a data frame easily: filter rows, select columns, create new columns, visualize your data, and calculate basic statistics.

The next big milestone is getting comfortable working with functions. Using functions will revolutionize how you program. At first, it may seem like writing functions takes way more time, but in the long term functions make your code more readable, less prone to bugs, and easier for you to come back to.

- Start with this Statistical Programming Course
  - Complete the homework assignments
- Read R for Data Science
- Experiment with functions, loops, and control flow in your code
- Learn Git and GitHub for version control
- If machine learning is needed for your research:
  - Learn basics of Pandas and NumPy in Python
  - Use Poetry to manage Python dependencies *(optional at this stage)*
  - Complete ML tutorials on Kaggle
  - Read through the scikit-learn User Guide
  - Participate in Kaggle learning competitions
  - Study how experts approach regression and classification tasks
- Consider learning SQL to efficiently manage and query large datasets

Note: Getting comfortable working with data is probably the hardest part of this whole path. Once you're past it, you'll gain a lot of momentum going forward.

Phase 2: Writing Cleaner Code

- Read Hadley Wickham's Advanced R and R Packages
  - Work towards creating well-documented functions to share with others
- Learn about non-standard evaluation for using tidyverse syntax in functions

- Develop debugging skills for functions
- Get comfortable with:
  - Git branches, rebasing, and pull requests
  - Command-line tools and terminal navigation
  - Simple bash scripting
- *(Optional)* Read *Grokking Algorithms* to understand basic data structures and algorithms
- Focus on refining your statistics skills simultaneously

Note: Phase 2 is especially useful if you share code with others, do a lot of data cleaning, or have other repetitive coding tasks.

Phase 3: Apply Your Skills to Research

- Balance coding skills with research questions and statistical knowledge
- Explore Bayesian frameworks that require computational power
- Learn to use:
  - High-performance computing clusters (if available)
  - AWS or Azure cloud computing

Note: These three phases are the most important for building a strong foundation in data science. The next phase is a collection of things that I found useful in my own research.

Phase 4: Optimization and Customization

- Switch to VS Code as your editor
- Experiment with Vim motions
- Customize your terminal environment
- Watch educational videos about interesting projects
- Participate in Kaggle projects and study other people's solutions
- For advanced software development:
  - Learn Docker containers (like a box to package everything you need to run a piece of software)
  - Master cloud computing services (AWS or Azure)
- Try Advent of Code if you want to challenge yourself or learn a new language

Data Manipulation in R and Python

Get Comfortable Working with Data in R and Python

Cleaning and Visualizing Data

The best place to start working with data is to clean and visualize it. You should calculate descriptive statistics on your dataset (for example with `summary()` in R or `df.describe()` in pandas) before running any statistical tests.

tidyverse (R)

Learning the tidyverse is a great way to write clean, readable R code for data manipulation. It also works seamlessly with ggplot2 for data visualization.

I recommend starting with this Statistical Programming in R course and the associated Kelsey Codes YouTube series. The course quickly introduces you to the tidyverse and ggplot2 and provides practical exercises.

Frequently Used tidyverse Functions

- `select()`: Choose specific columns
- `filter()`: Filter rows based on conditions
- `mutate()`: Create or transform variables
- `arrange()`: Sort data by column values
- `pivot_longer()`: Reshape wide data into long format
- `pivot_wider()`: Reshape long data into wide format
- `ggplot()`: Build layered plots, using `+` to add elements

You should also look into creating Quarto or R Markdown files to present your code in an aesthetic report.

pandas (Python)

Data manipulation in Python using pandas is conceptually similar to using base R or the Tidyverse. Once you're comfortable with one language, transitioning to the other becomes easier.

Frequently Used pandas Functions

- `df[['col1', 'col2']]`: Select specific columns
- `df[df['col'] == value]`: Filter rows based on conditions
- `df.assign()`: Create or modify columns
- `df.sort_values()`: Sort data by column values
- `df.melt()`: Reshape wide data into long format
- `df.pivot()` or `df.pivot_table()`: Reshape long data into wide format
- `df.plot()`, seaborn, or matplotlib: Create plots (analogous to ggplot2)

Writing Functions in R

Working with Functions and Packages in R

Topics Covered

1. Purpose of functions
2. Control flow with if/else
3. Iteration with `map()` and for loops
4. Grouping functions into packages

Why Use Functions

Q. Why use functions when you can just copy and paste your code?

A. Repeating yourself makes it more likely that you make a mistake while coding. Constantly repeating your code will make your scripts very long and hard for others to read and debug. Writing functions will also make it easier for you to reuse code that you wrote before.

Example

```
# Define function
fahr_to_kelvin <- function(temp) {
  kelvin <- ((temp - 32) * (5 / 9)) + 273.15
  return(kelvin)
}

# Call function
fahr_to_kelvin(32)
```

Advanced Function Usage

You can do more advanced things with functions, like piping the output of one function into another with the native pipe `|>` (R 4.1 or later):

```
kelvin_to_celsius <- function(temp) {
  celsius <- temp - 273.15
  return(celsius)
}
```

```r
fahr_to_kelvin(32) |> kelvin_to_celsius()
```

More information on functions can be found in this introduction to functional programming in R.

Control Flow in R

Q. What is control flow and why is it important?

A. Control flow is useful when you want certain code to run if certain conditions are met. It is typically used inside functions to change what code is executed based on the arguments that are passed in.

If...Else Control Flow Boilerplate

```r
if (condition) {
  # code to run if the condition is TRUE
} else {
  # code to run if the condition is FALSE
}
```

If...Else Control Flow Example

```r
x <- 8
if (x >= 10) {
  print("x is greater than or equal to 10")
} else if (x > 5) {
  print("x is greater than 5, but less than 10")
} else {
  print("x is less than or equal to 5")
}
```

More information on control flow can be found in the Statistical Programming in R lecture on control flow.

Loops and Iteration

Say you have a function and want to apply it to more than one thing, what do you do? One solution is to create a loop.

Steps

1. Define the function
2. Define a vector you want to apply the function to
3. Use `map()` from the purrr package to apply the function to each element in the vector
4. Alternatively, write a `for` loop that calls the function on each element

Example

```r
library(purrr)

square <- function(x) { return(x * x) }

# Create a vector of numbers
vector1 <- c(2, 4, 5, 6)

# map() returns a list; map_dbl() returns a numeric vector
map_dbl(vector1, square)

# The same thing with a for loop
for (x in vector1) {
  print(square(x))
}
```

More information on loops can be found in the Statistical Programming in R lecture on iteration.

Working with Packages

Once you have a set of related functions that you keep using in a workflow, it is very convenient to put them into a package, where you can group them together, document them for other users, and write tests. The `devtools` and `usethis` packages handle most of the setup for you. Writing your own packages is a pretty advanced topic, but I recommend starting with Hadley Wickham's *R Packages* book, which covers everything from creating a package directory to writing tests, documenting functions, and publishing your package on CRAN.

Debugging in R

Squash bugs fast

Learn to Debug R Code

Debugging is one of the most important skills to build once you start writing your own functions. It takes time to develop, but a good starting place is the debugging chapter of Hadley Wickham's *Advanced R*.

Debugging Functions

- **`traceback()`**

  Run this right after an error to see the sequence of function calls that led to it, so you know where to start looking.

- **`browser()`**

  Put this in the body of your function, and it will pause execution so you can interactively step through your function.

- **`debug()`**

  Flags a function so that the next time it runs, it pauses at the first line and lets you execute one statement at a time. Turn it off with `undebug()`.

- **`options(error = recover)`**

  After any error, lets you pick which function call to inspect at the moment things went wrong.

- **`message()`**

  Print statements are useful to pin down where you are during a function's execution. `message()` is better than `print()` for this because it can be silenced with `suppressMessages()`.

- **VS Code and RStudio debuggers**

  Both editors have really nice interactive debugging tools that are worth exploring.

Version Control Your Code

Learning Git and GitHub

Why You Should Learn Git and GitHub

Git tracks every change you make, allowing you to go back to a previous version of your code, similar to Track Changes or version history in Microsoft Word. GitHub, a cloud-based platform for hosting Git repositories, makes it simple to collaborate with others. Using features like branches, you can work on different parts of a project independently, then merge the changes. As long as you push your work to GitHub regularly, you never have to worry about losing your code.

General Git Workflow

```
# Start tracking Git history for a directory (folder)
git init

# Fetch and merge changes from the remote repository
git pull

# Check the status of your files
git status

# Stage the files you want to commit (you can specify files or use . for all changes)
git add .

# Commit your changes with a descriptive message
git commit -m "Describe what you changed or added"

# Push your commits to the remote repository
git push
```

Four Levels of Version Control

You don't need to learn everything at once. I think of Git in four levels: the basics with GitHub Desktop, working with branches, advanced Git features, and continuous integration. Each section below covers one level.

Level 1: Getting Started with GitHub

GitHub Desktop is great for 90% of what you'll be using Git for. Let's start with the basics:

- Create a GitHub account
- Download GitHub Desktop
- Download Git
- Work through the tutorial repository GitHub Desktop offers when you first open it, or the free courses on GitHub Skills

Two Ways to Start a New Project

Option 1: Clone Someone Else's Repository

To clone someone else's repository from GitHub:

- Copy the link to the repo
- Click File > Clone Repository in GitHub Desktop
- Paste the link and choose where you want to copy the repository locally

Option 2: Make Your Own Repository

To make your own version-controlled repository:

- Click File > New Repository in GitHub Desktop
- Give your repository a name and description
- Choose a local path
- Click "Create Repository"

Three terms you'll use constantly:

- **Committing:** saving changes locally with messages
- **Pushing:** transferring changes to GitHub
- **Pulling:** getting others' changes locally

More on committing, pushing and pulling

Level 2: Working with Branches

The next step is using branches. A branch is a separate line of development: a safe place to try out changes to your code before adding them to your main code.

Creating and Using Branches

1. In GitHub Desktop, click "Branch" in the top menu
2. Select "New Branch" and give it a name
3. Make your changes in this branch
4. Commit the changes to the branch

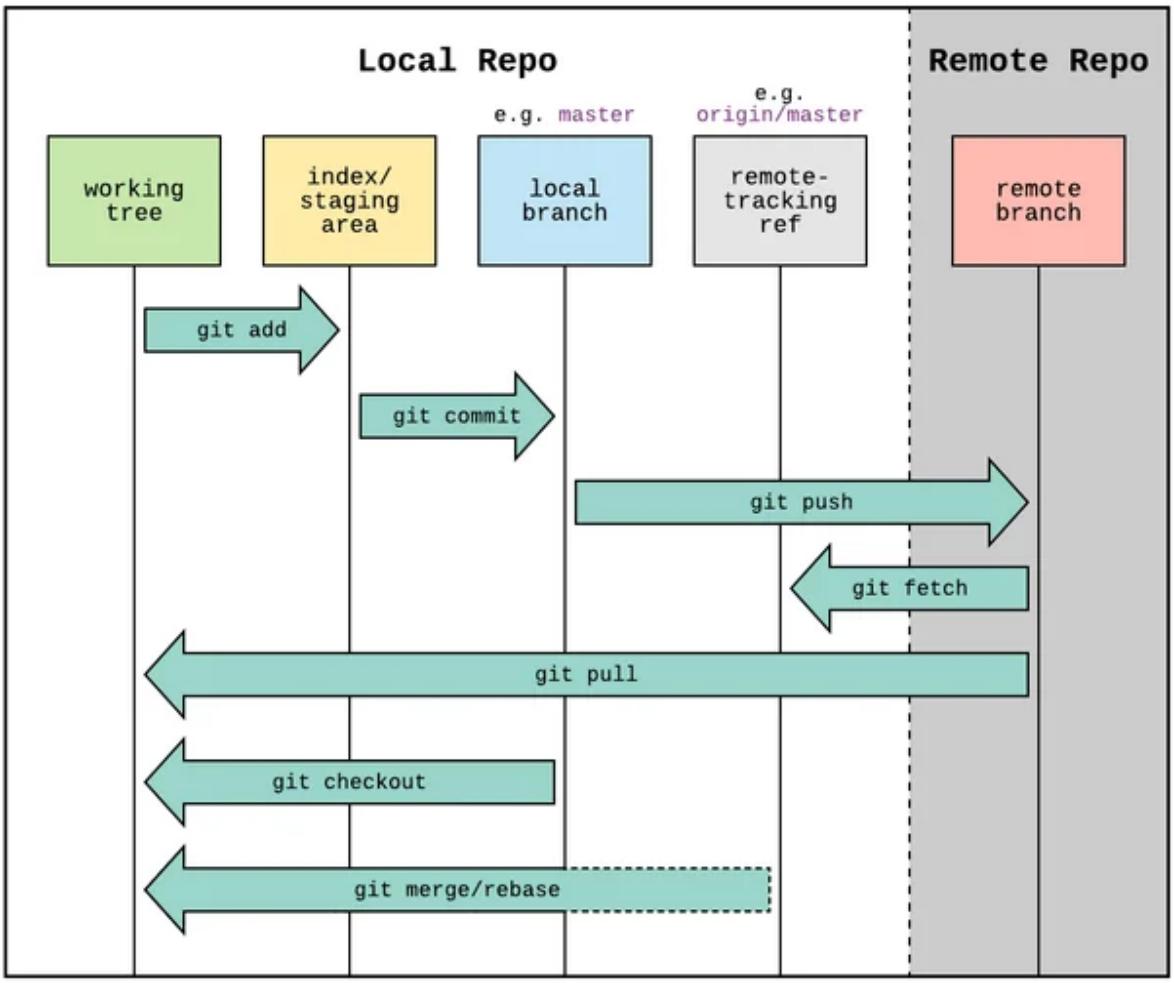


Figure 1: git workflow

5. Push the branch to GitHub

Once the changes you make to the new branch look good, you can merge them into your main branch, usually through a pull request (covered in Level 4). This is especially useful when you are working with other people who each want to make changes to the code and then combine them.

Level 3: Advanced Git Features

At this point, you understand the basics of Git, and the projects you work on are getting larger. Here are some advanced tips to help you stay organized.

.gitignore

A file that tells Git which files not to track. This should include large data files and anything containing passwords or API keys that you don't want on GitHub.

More information: freeCodeCamp's guide to .gitignore

Rebasing vs. Merging

I prefer rebasing over merging because it keeps a linear commit history, so when you want to go back to a previous change, it is easier to see what changed and in what order. Many teams prefer merging, so follow whatever your team uses. One rule always applies: never rebase a branch that other people have already pulled, because rebasing rewrites history.

More information: Atlassian's merging vs. rebasing tutorial

Squashing Commits

Before you rebase one branch onto another, it is super useful to squash (combine) your commits into as few as possible. Rebasing works by reapplying commits on top of a new base commit: it takes the commits from branch B and stacks them onto branch A. This is a lot easier with fewer commits, and it also makes it easier later on when you want to look through your commit history and see what you did.

More information: Git Tower's guide to squashing commits

Use Git in the Terminal

Using Git in your editor's built-in terminal is great because you never have to leave your code editor, and it opens up a bunch of advanced features that are harder to use in GitHub Desktop.

A useful tool for using Git in the command line is lazygit, which lets you use keyboard shortcuts instead of typing out entire Git commands.

Level 4: Continuous Integration

Once you start getting into the software side of coding, it is super useful to use advanced Git workflows and automation.

Pre-commit Hooks

Use pre-commit to ensure that all code changes are consistent in style, free of common mistakes, and do not include anything unintended before they are committed to your repository.

Pre-commit is a framework for managing and maintaining multi-language pre-commit hooks. These hooks run automatically before each commit and can catch issues like formatting errors, unused imports, and more.

You need to install and configure pre-commit in each repository you want it to run in:

```
pip install pre-commit   # install the tool once
# add a .pre-commit-config.yaml file to your repository, then:
pre-commit install       # activate the hooks for this repo
```

Once set up, the hooks will run automatically every time you make a commit, helping enforce code quality and consistency across your project.

Pull Requests

Another advanced feature is to combine pull requests with GitHub's branch protection rules, so that before anyone can add changes to your main branch, certain requirements (such as passing tests or an approving review) must be met.

1. Create a branch with your changes
2. Push the branch to GitHub
3. Go to the repository on GitHub and click "New pull request"
4. Select your branch to merge into main
5. Add reviewers and description
6. Create the pull request

GitHub Actions

GitHub Actions run automated workflows when something happens in your repository, such as a push or a new pull request. They are commonly used with pull requests to run tests before a branch is merged.

Command Line Tools

Get Fast with the CLI

Why Learn Command-Line Tools

After spending some time programming, you'll want to get faster at routine tasks, and you'll want to be able to easily explain what you did to someone else. I think graphical user interfaces are great when you are first learning an application, which is why I started the Version Control chapter with GitHub Desktop. But if you get comfortable with command-line tools, you can work a lot faster and easily have someone reproduce what you did without having to point and click.

Getting Good with Your Terminal

Learning to navigate your computer's file system with shell commands opens up several doors for you. The main benefit is being able to use an HPC (high-performance computing) cluster. If you are doing any sort of data analysis or machine learning, you will likely want to use a cluster at some point, and the command line is the main way to access one.

`pwd` (Print Working Directory)

This command shows your current location in the file system.

```
$ pwd
/Users/username/Documents
```

`ls` (List)

Lists all files and directories in your current location.

```
$ ls
Desktop  Documents  Downloads  Pictures
```

Add options like `-l` for detailed view or `-a` to show hidden files:

```
$ ls -la
```

`cd` (Change Directory)

Moves you to a different directory.

```
$ cd Documents
```

`cd ..`

Moves up one directory level (to the parent directory).

```
$ cd ..
```

File Management

`mv` (Move)

Moves files or directories to a new location, or renames them.

```
# Move a file
$ mv file.txt Documents/

# Rename a file
$ mv oldname.txt newname.txt
```

`cp` (Copy)

Copies files, or whole directories with `-r`.

```
$ cp file.txt backup.txt
$ cp -r project/ project_backup/
```

`mkdir` (Make Directory)

Creates a new directory.

```
$ mkdir results
```

`rm` (Remove)

Deletes files, or directories with `-r`. **There is no trash or undo**, so double-check before you run it.

```
$ rm old_file.txt
$ rm -r old_folder/
```

`less` and `cat`

`less` lets you scroll through a file (press `q` to quit); `cat` prints the whole file to the screen.

```
$ less data.csv
$ cat notes.txt
```

Package Management

`pip`

Package installer for Python. Use it to install Python libraries.

```
# Install a package
$ pip install numpy

# List installed packages
$ pip list
```

`brew` (Homebrew)

Package manager for macOS (and Linux). Use it to install software.

```
# Install software
$ brew install git

# Update Homebrew
$ brew update

# Upgrade installed packages
$ brew upgrade
```

Tips for Effective Terminal Use

- Tab to autocomplete (useful for completing file paths)
- Press the up arrow to recall previous commands
- Press Ctrl+R to search through your command history
- Use `clear` to clear screen
- Create aliases for frequently used commands in your `.bashrc` or `.zshrc` file

Introduction to Shell Scripts

It is also useful to learn writing shell scripts which can be used to automate tasks such as managing backups, batch-renaming files, setting up development environments, automating data processing pipelines, scheduling repetitive tasks with cron, and even simplifying complex command sequences into single, reusable scripts.

I typically create bash scripts when I want to rerun R scripts with different parameter combinations on a computing cluster.

Here is a good resource on learning bash scripting: freeCodeCamp's bash scripting tutorial

Python Environments

Using Poetry and Conda to Manage Dependencies

Going from R to Python

R is great, but Python has many packages that R does not, especially when it comes to machine learning (scikit-learn, PyTorch, etc.). One of the biggest sticking points I experienced moving from R to Python was managing virtual environments and dependencies. In R a lot of this is done behind the scenes, and you often do not have to worry about using an explicit package version for your code (the `renv` package is R's version of this). In Python, however, it is important to install specific versions of your dependencies for each project. A virtual environment gives each project its own isolated set of packages, so upgrading a library for one project won't break another, and anyone can recreate the exact setup you used to get your results. One tool I really like is Poetry.

Poetry Installation

- Download Python onto your computer (it comes with the `pip` package installer)
- Follow these instructions to install Poetry

General Workflow Using Poetry

```
# Set up Poetry for your project (creates pyproject.toml)
poetry init

# Optional: keep the virtual environment in a .venv folder inside your project
poetry config virtualenvs.in-project true

# Add packages (updates pyproject.toml and the poetry.lock file)
poetry add pandas

# Install everything listed in pyproject.toml
# (run this after cloning someone else's project)
poetry install

# Activate the virtual environment
eval $(poetry env activate)

# Or run a single command inside the environment without activating it
```

```
poetry run python my_script.py

# Update packages to the latest versions allowed by pyproject.toml
poetry update
```

Use pyenv if you need a different version of Python than the one on your system.

Alternative: uv is a newer, much faster tool that can replace pip, Poetry and pyenv in one.

Use Conda to Manage Dependencies Beyond Python

Conda is similar to Poetry, but it can install more than just Python packages, including R and system libraries such as CUDA. I think both are really good, and knowing a bit about both allows you to use whatever the rest of your team is using.

```
# Create a new conda environment
conda create --name myenv python=3.12

# Activate the environment
conda activate myenv

# Install packages in your conda environment
conda install package_name

# Export environment for reproducibility
conda env export > environment.yaml

# Create environment from file
conda env create -f environment.yaml

# Update all packages in the environment
conda update --all

# Remove an environment
conda env remove --name myenv
```

Tools for Machine Learning

scikit-learn, tidymodels, PyTorch, TensorFlow

scikit-learn

scikit-learn is great for traditional machine learning tasks like classification, regression, clustering, etc.

A good starting place is getting comfortable with NumPy, pandas, and Matplotlib in Python so that you can clean your data and split it into features (X) and a target (y). After that, learn the different ML algorithms and when to use them.

tidymodels

tidymodels is similar to scikit-learn but in R, and it follows tidyverse principles. Read more in the tidymodels getting-started guide.

Types of Machine Learning Algorithms

- **Supervised Learning**
  - Classification: Logistic Regression
  - Regression: Linear Regression
  - Both: Decision Trees, Random Forests, Gradient Boosting, Support Vector Machines (SVM)
- **Unsupervised Learning**
  - Clustering: K-Means Clustering, Hierarchical Clustering
  - Dimensionality Reduction: Principal Component Analysis (PCA)
- **Reinforcement Learning**
  - Game-playing agents (e.g., AlphaGo), robotics, and fine-tuning large language models (RLHF)

Data Splitting

Divide your data into the following subsets:

- **Training Data:** Used to fit the model.
- **Validation Data:** Used for tuning hyperparameters, comparing models, and detecting overfitting.
- **Testing Data:** Used once, at the end, to evaluate the final model's performance on unseen data.

Choosing the Right ML Algorithm

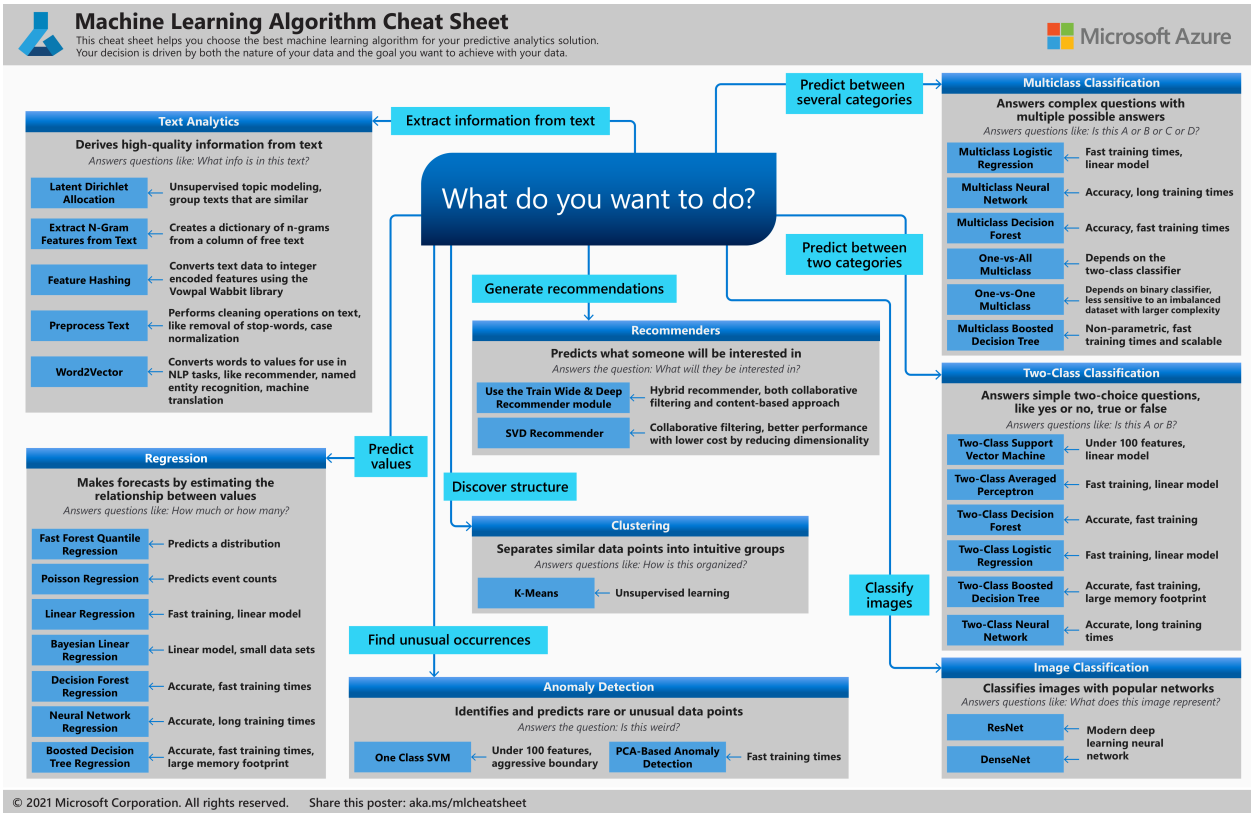


Figure 2: Machine Learning Algorithm Cheat Sheet (source)

Neural Networks

Neural networks are used for deep learning and are fairly advanced. The two most widely used deep learning frameworks are PyTorch and TensorFlow. PyTorch is more common in research and is a good default if you're starting out.

Basic Hyperparameters in Neural Networks

- **Learning Rate:** Controls how quickly the model updates its weights.
  - Too high – the model may overshoot and fail to converge (it bounces around).
  - Too low – the model converges very slowly and may get stuck in local minima.
- **Batch Size:** Number of training examples used in one update of model weights.
  - **Stochastic Gradient Descent (SGD):** Batch size = 1
  - **Mini-batch SGD:** Batch size > 1 but smaller than the full dataset
  - **Full-batch:** Entire dataset used (rare due to memory constraints)
- **Epoch:** One complete pass through the entire training dataset.
  - Example: 1,000 samples / batch size 100 = 10 iterations per epoch

Josh Starmer has a great course on YouTube that walks you through the principles of machine learning: the StatQuest machine learning playlist.

Getting Started with Bayesian Inference

Modeling Framework

Recommended Resources to Learn Bayesian Inference

Learn Theoretical Basis

This textbook builds up your understanding of Bayesian inference from fundamental principles of probability without getting too deep into the math. I recommend starting here and then quickly applying what you learn to your own data.
A Student's Guide to Bayesian Statistics by Ben Lambert

Introduction to Stan in R

Once you have a grasp of how Bayesian inference works, it's time to apply those methods to your own data. I recommend watching this video for an introduction to using Stan in R to implement Bayesian methods. Stan is a probabilistic programming language that lets you write and fit statistical models.
Bayesian Modeling with Stan and R

If you'd rather not write Stan code directly, the `brms` package lets you fit Stan models using familiar R formula syntax.

Introduction to NumPyro in Python

I recommend spending some time reading through the NumPyro documentation and then following along with the introductory tutorials.
NumPyro documentation

Programmer's Toolbox

Useful Resources for Programmers

New Tools

Switching from a tool you are comfortable with to something new can often feel overwhelming. You'll feel unproductive and frustrated, but if you plan on coding in the long term, this initial time investment will pay off hugely.

Why Learn VS Code

VS Code is the industry-standard code editor, and for good reason. It is super fast, lightweight, and highly customizable. RStudio is great for R, but if you ever want to branch out and learn a new programming language, VS Code will set you up perfectly for this. Another great addition to VS Code is its integration with GitHub Copilot, which can be super useful for code completions.

Posit's Positron is also worth a look: a VS Code-based editor built for data science that will feel familiar to RStudio users.

- Download VS Code

My VS Code Setup (macOS)

- Vim-like keyboard shortcuts: keyboard_shortcuts_bm.JSON
- General VS Code settings for coding in R and Python: vscode_settings_bm.JSON

What Are Vim Motions?

Vim motions are a set of key bindings that let you quickly navigate and edit text without using your mouse. Basically, they allow you to keep your hands on the keyboard at all times. For example, with Vim motions you can quickly copy and paste a line of code, delete the code inside parentheses, and so much more.

Why Learn Vim Motions

Learning Vim motions is very tough at first, even though the controls become intuitive with practice. It will likely reduce your productivity for a bit until you get comfortable. One common argument for learning Vim

is that it will make you a faster programmer. I would argue that the best reason to learn Vim is that it reduces the time between when an idea pops into your mind and when your fingers can start to type it out.

- Vim extension for VS Code
- Learn Vim motions with Vim Hero
- You can also set up Vim motions in RStudio
- I also use Vim motions in my browser with the Vimium Chrome extension
- I recommend mapping your Caps Lock key to Escape so you can quickly switch from insert mode back to normal mode

Autoformat Code in R

- VS Code extension that autoformats your code when you save a file
  - Air - R Language Support
- R package to autoformat your code in RStudio
  - The `styler` package

Useful Tools for Developers

Raycast (macOS)

- Use shortcuts to launch apps
- Quickly switch between your browser and code editor

AeroSpace (macOS)

- Tiling window manager for macOS
- Easily manage windows and workspaces

Ghostty (macOS and Linux)

- Really fast terminal
- Native look and feel, with tabs and split panes
- Very customizable

Obsidian

- Markdown-based note-taking
- Great global search to quickly find notes
- Has great plugins that allow you to write formatted code

Further Learning

Useful Courses and Fun Content

Useful Courses

Name	Description
Statistical Programming in R	Introductory course teaching you the basics of using the tidyverse for data analysis
Kelsey Codes (YouTube)	Video series that goes along with the course above
R for Data Science	Free book on data analysis with the tidyverse
Data Wrangling in Python	Teaches data wrangling in Python using pandas
Atlassian Git Tutorials	Reference for common Git commands
Advanced R	Deep dive into advanced R programming concepts
R Packages (2e)	Guide to creating and maintaining R packages
The Missing Semester of Your CS Education	Practical tools CS courses skip: the shell, editors, Git, and debugging
Grokking Algorithms	Illustrated guide to understanding algorithms
Scikit-Learn ML from Start to Finish (Kaggle)	Machine learning tutorial using scikit-learn, from start to finish
StatQuest Machine Learning (YouTube)	Clear, visual explanations of machine learning concepts
QCBS R Workshop Series	Useful for learning cluster analysis, GAMs, GLMs, etc.
Bayes Rules!	Free introductory book on Bayesian modeling in R

Fun Content

Name	Description
ThePrimeagen - YouTube	Content on learning Git and Vim motions
Theo - t3.gg - YouTube	Keep up to date on latest tech developments
Vim in VS Code - YouTube	Helps you set up Vim motions in VS Code